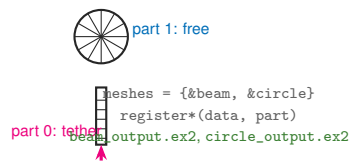
TWO OR MORE MESHES

Parts in `IBFEMethod`: one mesh file per body or one file placed twice, per-part registration and output, a plate driven into a free disc (IBFE/08), the 2020 clap-and-fling wings with their kinematics corrected, and the two-body technique of the 2018 coral example



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part IV • Chapter 21

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. give `IBFEMethod` several meshes (“parts”), register stress and force functions part by part, and write one ExodusII file per part, with the prefix mechanism that names them;
2. choose between two mesh files, one file read twice and placed by coordinate mappings, and one mesh with subdomains, and say when each is right;
3. build and run IBFE/08, a tethered plate driven into a free disc, and read from its two output files how close the two bodies get and why the disc keeps moving;
4. read the 2020 clap-and-fling wing example: two wings, two target functions, the same stress, $\kappa_s = 10^{13}$, and the half-stroke switch in its kinematics that leaves the target’s velocity discontinuous; write the corrected rigid-wing kinematics the 2020 slide promised;
5. describe the 2018 two-coral technique (CAD to a hex mesh, one body placed twice, a bell-like bending motion) and reproduce it with a neutral body;
6. look at several parts at once in VisIt, and restart a multi-part run without damaging its files.

> Everything of chapters 17–20 was one mesh. Here the only new word is *part*: an index that every registration, every data manager and every output file carries.

WHERE THIS CHAPTER COMES FROM

- "6-IBFE: Two Meshes" (2015, edited 2018; 6 slides): "Random info", the ca.-2016 two-mesh code (`getExodusIIFilename("mesh1")`, `vector<Mesh*> meshes`, per-part registration, two `AutoPtr<ExodusII_IO>`), and "Laura's Notes" on the constructor and `postprocess_data`. The 2020 "Plans for June" deck repeats the code as the "ca. 2016 approach".
- "7-IBFE: Two Boundaries: Flapping wings and beam hits circle" (2020; 20 slides): the 3D clap-and-fling wings (Trelis meshes, prescribed sweep and angle of attack, a target function per wing) and the 2D beam driven into a circle, with the current `std::vector<MeshBase*>` and `std::unique_ptr` code. Example IBFE/08 is the second; the first is `examples/legacy/UNC-Examples/7-IBFE-TwoWings`.
- The 2016 example `Example_IBFE_TetherForceMultiplePlate2D` (course set 2016-11): the "... multipleplate2d" of the 2015 slide: one plate mesh read twice, placed by two coordinate mappings, each copy with its own target function.
- "2-IBFE Detailed Example using Target Points in 3D" (MATH 769, 2018; 14 slides, private): two pulsing corals, Fusion 360 → Bolt hex mesh → two copies placed by coordinate mappings → a bell-like bending motion with a contraction and an expansion phase. The deck ends before the code it promises; the coral research model is not in the public collection. Its technique is taught here with a neutral body (slide 22).
- New here: the `IBFEMethod` constructor and `AppInitializer::getExodusIIFilename(prefix)` from the source; the registration table for several parts; IBFE/08's measured results and figures; the wing kinematics analysed (continuous position, discontinuous velocity at each half-stroke; the thickness sheared) and corrected; restart with several parts; VisIt with several databases; upstream ex6 and ex8 as further patterns.

A PART IS AN INDEX

```
IBFEMethod(const std::string& object_name, const SAMRAI::tbox::Pointer<SAMRAI::tbox::Database>& input_db,
           libMesh::MeshBase* mesh, int max_levels, bool register_for_restart = true,
           const std::string& restart_read_dirname = "", unsigned int restart_restore_number = 0);      // one mesh
IBFEMethod(const std::string& object_name, const SAMRAI::tbox::Pointer<SAMRAI::tbox::Database>& input_db,
           const std::vector<libMesh::MeshBase*>& meshes, int max_levels, bool register_for_restart = true,
           const std::string& restart_read_dirname = "", unsigned int restart_restore_number = 0);      // several
```

where the part appears	what it means
the order of `meshes`	part 0 is `meshes[0]`, part 1 is `meshes[1]`, ...; the one-mesh constructor is the vector of length one
`register*(data, part)`	every registration function of chapter 19 takes the part as its last argument (default 0): stress functions, body, surface and pressure forces, coordinate mapping, initial velocity
`getFEDataManager(part)`	each part has its own `FEDataManager`, equation systems, and quadrature settings; `->getEquationSystems()` is what its ExodusII writer writes
one `ExodusII_IO` per part	each writer is built on its own mesh and writes its own file (slide 8)
`elem->subdomain_id()`	is *not* a part: subdomains live inside one mesh (chapter 18) and are told apart inside a stress function

The input file does not know about parts: one `IBFEMethod` block serves all of them (`IB_point_density`, the kernel, `split_forces` apply to every part).

THREE WAYS TO GET TWO BODIES

way	how	used by
A. two mesh files	`beam_mesh.read("beam.e")`, `circle_mesh.read("circle.e")`, two entries in `meshes`	IBFE/08; the two wings (`pos_wing.e`, `neg_wing.e`); upstream ex6 and ex8 build theirs in the program
B. one file, read twice, placed twice	two `Mesh` objects read the same file; a coordinate mapping per part puts each copy where it belongs (`X = s + shift`, or a rotation)	the 2016 two-plate example (slide 13); the 2018 corals (slide 22)
C. one mesh, two subdomains	one part; the mesher's physical groups or Exodus blocks give the elements ids; the stress function tests `elem->subdomain_id()`	a stem and a leaf, a wall and its stiff band; chapter 18, slide 26

- A and B are parts: each body can have its own forces, its own output file, its own stress; the bodies do not share nodes and can touch, separate or pass each other. The fluid between them is what keeps them apart (IBFE/08's plate never touches the disc, slide 11).
- C is one body with two materials: the two regions are joined, strain passes from one to the other, and there is one output file. Use it for a body with a stiff and a soft region, not for two bodies.
- B saves a mesh file and guarantees identical bodies; A lets the bodies differ. Both need a coordinate mapping only if the file is not already where the body should start (chapter 22 has the mapping in full).

IBFE/08: A PLATE DRIVEN INTO A FREE DISC

"In this example, a 2D beam is propelled towards a 2D circle. Target points drive the beam for the first 1 second, and then the beam is free to coast. The circle is untethered and moves due to the approaching beam" (2020, slide 15).

- Domain 2×2, periodic; two levels, $32 \to 128$ cells; fluid $\rho = 1$, $\mu = 0.01$.
- Part 0, the plate: 0.0312 wide, 0.5 tall, upright, centre $(0, -0.5)$: `beam.e`, 98 nodes, 126 `TRI3`, made with Trelis.
- Part 1, the disc: radius 0.25, centre $(0, 0.5)$: `circle.e`, 977 nodes, 1851 `TRI3`. Its bottom starts 0.5 above the plate's top.
- Both neo-Hookean, $\mu = 10$ (`SHEAR_MODULUS`); the plate's tether moves up at `TETHER_SPEED = 0.25` until `TETHER_END_TIME = 1.0`, $\kappa_s = 10^6$; the disc has no tether.
- 8000 steps of `DT = 5e-4`; 81 frames, one every 0.05 (`viz_dump_interval = int(.05/DT)`, the 2015 slide's tip); about two minutes on four cores.

```
// the tether on the plate (part 0) only
void beam_tether_force_function(VectorValue<double>& F, ...,
  const libMesh::Point& X,
                                  const libMesh::Point& s, ...,
  double time, void* /*ctx*/)
{
    libMesh::Point s_target = X;                    // no force
  by default
    if (time < tether_end_time)
    {
        s_target(0) = s(0);
        s_target(1) = s(1) + tether_speed * time; // up at 0.25
    }
    F = kappa_s * (s_target - X);
}
```

The 2020 slide 17 is the same with `.25*time` and `time < 1` written in.

IBFE/08 (2): THE TWO MESHES AND THE PARTS

```
// Each part gets its own ExodusII file: viz_IB2d/beam_output.ex2 and viz_IB2d/circle_output.ex2.
const bool uses_exodus = dump_viz_data && !app_initializer->getExodusIIFilename().empty();
const string beam_exodus_filename = app_initializer->getExodusIIFilename("beam_");
const string circle_exodus_filename = app_initializer->getExodusIIFilename("circle_");
...
// Read the two meshes. libMesh chooses the reader from the file name; .e is an ExodusII file.
// The order of the meshes in the vector sets the part numbers: the plate is part 0 and the disc is part 1.
Mesh beam_mesh(init.comm(), NDIM);
beam_mesh.read(input_db->getStringWithDefault("BEAM_MESH_FILENAME", "beam.e"));
beam_mesh.prepare_for_use();
Mesh circle_mesh(init.comm(), NDIM);
circle_mesh.read(input_db->getStringWithDefault("CIRCLE_MESH_FILENAME", "circle.e"));
circle_mesh.prepare_for_use();
std::vector<MeshBase*> meshes = { &beam_mesh, &circle_mesh };
...
Pointer<IBFEMethod> ib_method_ops = new IBFEMethod("IBFEMethod", app_initializer->getComponentDatabase("IBFEMethod"),
                                                   meshes, max_levels, /*register_for_restart*/ true,
                                                   restart_read_dirname, restart_restore_num);
...
// Configure the IBFE solver. The last argument of each register call is the part number.
// Both parts get the same stress functions; only the plate (part 0) gets the tether force.
ib_method_ops->registerLagBodyForceFunction(beam_tether_force_data, 0);
ib_method_ops->registerPK1StressFunction(PK1_dev_stress_data, 0);
ib_method_ops->registerPK1StressFunction(PK1_dil_stress_data, 0);
ib_method_ops->registerPK1StressFunction(PK1_dev_stress_data, 1);
ib_method_ops->registerPK1StressFunction(PK1_dil_stress_data, 1);
ib_method_ops->initializeFEEquationSystems();
EquationSystems* beam_equation_systems = ib_method_ops->getFEDataManager(0)->getEquationSystems();
EquationSystems* circle_equation_systems = ib_method_ops->getFEDataManager(1)->getEquationSystems();
```

IBFE/08 (3): ONE EXODUSII WRITER PER PART, AND THE RESTART RULE

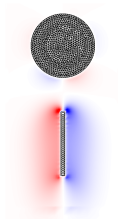
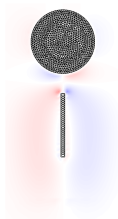
```
// Set up visualization plot file writers: one ExodusII writer per part.
std::unique_ptr<ExodusII_IO> beam_exodus_io = uses_exodus ? std::make_unique<ExodusII_IO>(beam_mesh) : nullptr;
std::unique_ptr<ExodusII_IO> circle_exodus_io = uses_exodus ? std::make_unique<ExodusII_IO>(circle_mesh) : nullptr;
// A restarted run adds its time steps to the existing ExodusII files. Only processor 0 writes the files, so only
// processor 0 reopens them: libMesh 1.7.8 reopens them on every processor when told to append, and with more than
// one process the files are then damaged (they can no longer be read).
if (uses_exodus)
{
    const bool from_restart = RestartManager::getManager()->isFromRestart();
    beam_exodus_io->append(from_restart && IBTK_MPI::getRank() == 0);
    circle_exodus_io->append(from_restart && IBTK_MPI::getRank() == 0);
}
...   // in the time loop, at every dump:
beam_exodus_io->write_timestep(beam_exodus_filename, *beam_equation_systems, iteration_num / viz_dump_interval + 1, loop_time);
circle_exodus_io->write_timestep(circle_exodus_filename, *circle_equation_systems, iteration_num / viz_dump_interval + 1, loop_time);
```

- `getExodusIIFilename(prefix)` returns `viz_dump_dirname + "/" + prefix + exodus_filename` (`AppInitializer.cpp`): the prefix is glued to the file name (`output.ex2` unless the `Main` database sets `exodus_filename`) with no separator. The course wrote `"mesh1"` and got `mesh1output.ex2`; the example writes `"beam_"` and gets `beam_output.ex2`. Two parts with no prefix would write the same file twice.

IBFE/08 (3B): DUMPS AND RESTARTS WITH TWO PARTS

- One `write_timestep` per part at every dump, with the same time-state number; both files hold 81 states here (0.6 MB for the plate, 5.2 MB for the disc). A part written at a different interval would need VisIt's database correlation to line up (slide 25).
- `register_for_restart = true` in the constructor saves and reloads the finite element data of every part with the fluid's restart files; a restarted run then appends its new states to both ExodusII files. The shipped input has `restart_dump_interval = 0`; `int(0.5/DT)` writes a restart every half time unit.
- The restart rule is IBFE/08's own finding (0.16.0 and 0.19.0): IBAMR's examples pass `append(from_restart)` on every rank, and a restart on more than one process leaves files that VisIt cannot open. Only rank 0 writes, so only rank 0 reopens.
- The same rule holds with one part; it only shows up more often with several, because multi-part runs are the long ones that get restarted.

IBFE/08 (4): WHAT HAPPENS


$t = 0$: gap 0.500 $t = 1$: released; gap 0.269 $t = 2$: gap 0.156 $t = 4$: gap 0.094

- Vorticity ± 5 on the fluid grid; the two meshes, each displaced by its own `dX`. Window $[-0.45, 0.45] \times [-0.85, 0.85]$.
- The plate is pushed up for one second and coasts; the disc moves because the fluid the plate pushed moves it. The vorticity pair at the plate's top is the wake of a blunt body; the disc deforms hardly at all (its width stays 0.500).

IBFE/08, IBAMR 0.19.0 (`/root/work/0.19.0/ibfe08_final`); `wsl/final/visit_ch21.sh`.

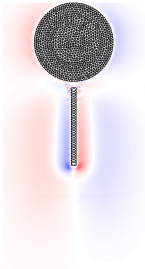
IBFE/08 (5): THE NUMBERS, AND WHY THE DISC KEEPS MOVING

From the two ExodusII files (plate top: its highest nodes; disc bottom: its lowest):

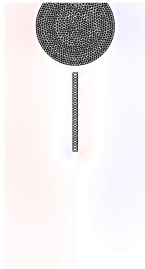
time	plate top y	disc bottom y	gap	
0	-0.250	0.250	0.500	
1.0	0.000	0.270	0.269	the plate is released; its top is where the target put it
2.0	0.146	0.301	0.156	
3.0	0.222	0.335	0.113	
4.0	0.274	0.368	0.094	about six fine grid cells

- While tethered the plate follows its target to three digits. Released, it keeps rising more and more slowly, and the gap keeps closing, but the plate does not reach the disc: the disc is moving away too.
- Why: the tether pushes the plate, and through it the fluid, upward. In a periodic box that momentum has nowhere to go; after $t = 1$ the average upward velocity of the whole fluid stays at 0.016 (0.0162 at $t = 1$, 0.0159 at $t = 4$, from the VisIt files). The disc sits in the column of fluid above the plate and rises at about 0.033. A periodic box is not a tank: chapter 7's boundary conditions, or a longer box, change this.
- `TETHER_END_TIME = 2.0` (the README's "Things to try"): the plate's top reaches $y = 0.250$ at $t = 2$ and the gap closes to 0.040 at $t = 2.5$, about 2.6 fine cells, then stays near 0.045: the fluid between them keeps them apart, and the disc is squeezed a little (width 0.5006). Next slide.
- 0.16.0 and 0.19.0 agree to 2×10^{-8} in the node positions.

IBFE/08 (6): PUSHING LONGER



`TETHER_END_TIME = 2`, $t = 2.5$: gap 0.040



$t = 4$: gap 0.045

- The two bodies come within three fine grid cells and stay there. There is no contact model in IBFE: the fluid film between them, resolved by two or three cells, carries the load. Closer than that, the two bodies' quadrature points spread force into the same cells and the film can no longer thin; the point method of Part II behaves the same way.
- Real contact (a valve leaflet on its seat, two corals brushing) needs a thinner film resolved by a finer grid, or a penalty between the surfaces written as surface forces (chapter 19, slide 24) that test the other body's position through `ctx`.
- The disc's width grows to 0.5006 at $t = 2$: it is being pressed; its own neo-Hookean stress pushes back.

`/root/work/0.19.0/ibfe08_push2`, IBAMR 0.19.0.

THE 2016 TWO-PLATE EXAMPLE: ONE FILE, READ TWICE, PLACED BY COORDINATE MAPPINGS

`Example_IBFE_TetherForceMultiplePlate2D` (course set 2016-11; the "... multipleplate2d" of the 2015 slide):

```
void coordinate1_mapping_function(libMesh::Point& X, const libMesh::Point& s, void* /*ctx*/)
{   X(0) = s(1)+.2;  X(1) = s(0);  return; }          // the plate turned upright and put at x = +0.2
void coordinate2_mapping_function(libMesh::Point& X, const libMesh::Point& s, void* /*ctx*/)
{   X(0) = s(1)-.2;  X(1) = s(0);  return; }          // its twin at x = -0.2
void target1_force_function(..., const libMesh::Point& X, const libMesh::Point& s, ..., double time, ...)
{   libMesh::Point s_dump;
    if(time<1)      { s_dump(0)=s(1)+.2+.5*time;        s_dump(1)=s(0); }   // move right at 0.5
    else if(time<2) { s_dump(0)=s(1)+.2+.5+.5*(1-time); s_dump(1)=s(0); }   // and back
    else            { s_dump(1)=X(1); s_dump(0)=X(0); }
    F = kappa_s*(s_dump-X); return; }
// target2_force_function: the mirror image, s(1)-.2-.5*time, ...
...
mesh1.read("IBFE_Mesh2D_128.mat");  mesh1.prepare_for_use();
mesh2.read("IBFE_Mesh2D_128.mat");  mesh2.prepare_for_use();
vector<Mesh*> meshes(2);  meshes[0] = &mesh1;  meshes[1] = &mesh2;
...
ib_method_ops->registerInitialCoordinateMappingFunction(coordinate1_mapping_function, 0);
ib_method_ops->registerInitialCoordinateMappingFunction(coordinate2_mapping_function, 1);
ib_method_ops->registerLagBodyForceFunction(target1_force_data, 0);
ib_method_ops->registerLagBodyForceFunction(target2_force_data, 1);
```

THE 2016 TWO-PLATE EXAMPLE (2): WHAT THE MAPPINGS AND THE TARGETS DO

- The mapping swaps the axes, so the horizontal plate of chapter 17 stands upright, and shifts each copy sideways; the two plates then move apart at 0.5 for a second and back for a second: a pair of paddles. The target functions must apply the same mapping to `s` ("`s(1)+.2`"), because `s` is the *file's* reference position, not the mapped one: chapter 22 makes this precise.
- Each copy has its own target function, registered on its own part; the two functions differ only in the sign of the shift and of the motion. One function with the part's shift passed through `ctx` (chapter 19, slide 23) would do the same with less code.
- At $t = 0$ the plates stand 0.4 apart; at $t = 1$, 1.4 apart; at $t = 2$ they are back. After $t = 2$ the target is the current position, so the tether does nothing and the plates are free (exercise 4).
- This is the pattern the two-coral deck followed two years later with a 3D hex mesh (slide 22).

THE 2015 AND 2020 NOTES, AND THE CODE THEY CORRECTED

"Random info" (2015):

- "Example . . . multipleplate2d"; "VisIt → make first after adding dumps.visit stuff" (the Draw button, so that the fluid's time axis exists before the Exodus databases are added);
- "copies of everything with coordinate mappings in main.C";
- "`viz_dump_interval = int(0.05/DT)` helpful in input2d/input3d in general" (IBFE/08 uses it: a dump every 0.05 whatever `DT` is).

"Laura's Notes" (2015): "Mesh needs to be changed to meshes on this line", the constructor with `meshes`; "also need to change meshes here as well as equation systems" in `postprocess_data`, which took one mesh and one `EquationSystems`: with two parts it wrote one of them.

The 2020 deck's "Other changes from earlier versions":

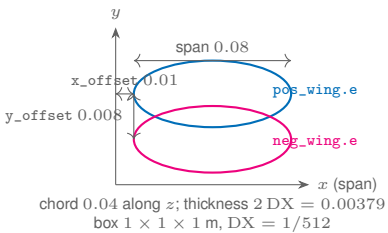
```
// these lines
AutoPtr<ExodusII_IO> mesh1_exodus_io(uses_exodus ? new
  ExodusII_IO(mesh1) : NULL);
// should be changed to these lines
std::unique_ptr<ExodusII_IO> mesh1_exodus_io(uses_exodus ? new
  ExodusII_IO(mesh1) : NULL);
```

and, in `postprocess_data`, "you may need to update the FEBase and QBase lines":

```
unique_ptr<FEBase> fe(FEBase::build(dim, F_dof_map.
  variable_type(0)));
unique_ptr<QBase> qrule = QBase::build(QGAUSS, dim, FIFTH);
```

- `vector<Mesh*>` (2015) became `std::vector<MeshBase*>` (2020): the constructor takes the base class.
- A post-processing function for several parts loops over `getFEDataManager(part)` and writes one line per part; chapter 22's flow tank does this for its tether force.

THE 3D CLAP-AND-FLING WINGS (2020): THE SET-UP



Redrawn from the 2020 slide 3 (which wrote $\mathrm{DX} = 1/528 = 0.00189$; the input's $N = 8$, four levels at ratio 4 give 512 cells and 0.00195).

- "Two elliptical wings performing a simple clap and fling motion. Two separate meshes are uploaded and moved by target forces with different motions. The wings were created using Trelis. This simulation takes a long time to run and generates a lot of data."
- "I used the trial version of Trelis to make 2 elliptical wings with the right dimensions, rotated and translated them to the correct initial positions. The getting-started tutorial was all I needed to learn enough in under an hour. I exported the meshes as Exodus `.e` files."

THE WINGS (2): FLUID, MATERIAL, TETHER, AND THE REFERENCE AXES

- Water (`RHO = 1000`), $f = 1$ Hz, and the viscosity set from a target Reynolds number: `MU = RHO*(WSPAN/2.0)*((2.0/3.0)*2.0*3.14*WSPAN*FREQUENCY)/RE` with `RE = 4`: a tiny insect's regime (chapter 8b).
- Compressible neo-Hookean wings with $E = 2$, $\nu = 0.3$ (IBFE/10's law); `KAPPA_S = 1.0e13` ("`//5.0e4`" commented out): the wings are meant to be rigid.
- "Let `s(2)` be position along chord and `s(0)` be position along span in initial configuration; `s(2) = 0` in the middle of the wing" (2020). Two target functions, one per part, mirror images; the same two stress functions registered on parts 0 and 1 (the deck's slide 12: "applied to mesh 0 / mesh 1 / both meshes").
- `DT = 1e-5` with `GROW_DT = 1e-2`: a hundred thousand steps per flapping cycle at $\mathrm{DX} = 1/512$ in 3D, which is why "this simulation takes a long time to run"; no run of it was made for this chapter.

THE WINGS (3): THE KINEMATICS, AND A TARGET FUNCTION PER WING

```
static double kappa_s = 1.0e13; //5.0e4;
static const double freq = 1.0;                    // frequency in hz of flapping
static const double aoa_not = PI/2.0;              // angle of attack at mid-stroke
static const double flap_period = 1.0/freq;
static const double B = PI/4.0;                    // pitch amplitude
static const double sweep_angle = 2.0*PI/3.0;      // sweep amplitude
void target1_force_function(VectorValue<double>& F, ..., const libMesh::Point& X, const libMesh::Point& s, ..., double time, ...)
{
    libMesh::Point s_dump;  double kappa;
    double sweep_angle_pos = 0.5*sweep_angle*(1-cos(2*PI*freq*time));   // sweep: 0 -> 120 deg -> 0 each cycle
    double aoa_pos = aoa_not-B*sin(2*PI*freq*time);                     // pitch: 90 deg -/+ 45 deg
    double chord_length = 0.04;  double offset = 0.01;                 // (offset is never used)
    kappa=kappa_s;  s_dump = s;
    double flap_no = floor(time/flap_period);  double curr_time = time-(flap_no*flap_period);
    if (curr_time<flap_period/2.0){  //downstroke
        s_dump(0) = s(0)*cos(sweep_angle_pos)+((s(2)+0.5*chord_length)*cos(aoa_pos)+s(1))*-sin(sweep_angle_pos);
        s_dump(1) = ((s(2)+0.5*chord_length)*cos(aoa_pos)+s(1))*cos(sweep_angle_pos)+s(0)*sin(sweep_angle_pos);
        s_dump(2) = s(2)*sin(aoa_pos);
    }
    else { //upstroke
        s_dump(0) = s(0)*cos(sweep_angle_pos)+(s(2)*cos(aoa_pos)+s(1))*-sin(sweep_angle_pos);
        s_dump(1) = (s(2)*cos(aoa_pos)+s(1))*cos(sweep_angle_pos)+s(0)*sin(sweep_angle_pos);
        s_dump(2) = s(2)*sin(aoa_pos);
    }
    F = kappa*(s_dump-X);
}
// target2_force_function: the same with sweep_angle_neg = -0.5*sweep_angle*(...) and aoa_neg = aoa_not+B*sin(...)
```

THE WINGS (4): WHAT THE KINEMATICS DO, AND THE HALF-STROKE SWITCH

The intended motion: the chord vector $(0, 0, s_2)$ is pitched by $\alpha(t)$ into $(0, s_2\cos\alpha, s_2\sin\alpha)$, the thickness s_1 is added along y, and the result is swept by $\phi(t)$ about the z axis:

$$\mathbf{X} = \begin{pmatrix} s_0\cos\phi - (s_2\cos\alpha + s_1)\sin\phi \\ (s_2\cos\alpha + s_1)\cos\phi + s_0\sin\phi \\ s_2\sin\alpha \end{pmatrix},$$

with $\phi = \frac{1}{2}\cdot\frac{2\pi}{3}(1-\cos 2\pi f t)$ and $\alpha = \frac{\pi}{2} \mp \frac{\pi}{4}\sin 2\pi f t$.

- The half-stroke switch. On the downstroke the code pitches about a point half a chord away (`s(2) + 0.5*chord_length`); on the upstroke about `s(2)` itself. At the switch ($t = \frac{1}{2}, 1, \ldots$) $\sin 2\pi f t = 0$, so $\alpha = \frac{\pi}{2}$, $\cos\alpha = 0$, and the two formulas give the *same position*: no jump in the target. But their time derivatives differ by $\frac{c}{2}\sin\alpha\,\dot\alpha\,(-\sin\phi, \cos\phi, 0)$ with $\dot\alpha = \mp\frac{\pi}{4}\cdot 2\pi f$ (the pitch amplitude is $\pi/4$): the target's velocity jumps by $\frac{c}{2}|\dot\alpha| = 0.02\cdot\frac{\pi^2}{2} \approx 0.10$ m/s at every half-stroke. With $\kappa_s = 10^{13}$ the plate follows and the fluid sees an impulsive acceleration: a force spike at every half-stroke, and a `DT` of 10^{-5} to survive it.

THE WINGS (5): THE SHEAR, THE PIVOT, AND THE UNUSED VARIABLES

- The shear. The thickness s_1 is added along y *before* the sweep but is not pitched with the chord, so at $\alpha \neq \frac{\pi}{2}$ the wing's cross-section is sheared, not rotated: "these equations work for an infinitely thin wing, but shear a wing that has some thickness. Corrected equations forthcoming" (2020, slide 6). At $2\,\mathrm{DX}$ thickness the shear is small; the tether at 10^{13} enforces it anyway, and the compressible material absorbs it.
- The pivot. Clap-and-fling wings pitch about their leading edge; the leading edge is the same physical edge on both strokes, so one pivot, fixed in the wing, is the kinematics. The switch between two pivots was a way to keep the wing's leading edge in the right place on each stroke; the corrected version below does that by choosing the pivot once and letting α carry the wing through.
- `offset = 0.01` is computed and unused; `flap_no` and `curr_time` are used only for the switch; `kappa = kappa_s` is a copy that is never changed. These are leftovers of a version that softened the tether near the wing's edge (the `//5.0e4`), which is a reasonable thing to do: a tether stiff in the middle and soft at the edge lets the tip flex (chapter 19, slide 26).

THE WINGS (6): THE CORRECTED RIGID-WING KINEMATICS

A rigid wing is a rotation of its reference shape: pitch the cross-section as a whole, then sweep. With the pivot p along the chord (the leading edge, $p = -c/2$, or the mid-chord, $p = 0$) chosen once:

$$y_1 = (s_2 - p)\cos\alpha - s_1\sin\alpha, \quad z_1 = (s_2 - p)\sin\alpha + s_1\cos\alpha, \qquad \mathbf{X} = \big(s_0\cos\phi - y_1\sin\phi,\ \ y_1\cos\phi + s_0\sin\phi,\ \ z_1\big).$$

```
static const double pivot = -0.5 * chord_length;   // pitch about the leading edge, the same edge on both strokes
void wing_target(libMesh::Point& s_target, const libMesh::Point& s, const double phi, const double alpha)
{
    const double y1 = (s(2) - pivot) * std::cos(alpha) - s(1) * std::sin(alpha);   // the cross-section, pitched as a whole
    const double z1 = (s(2) - pivot) * std::sin(alpha) + s(1) * std::cos(alpha);
    s_target(0) = s(0) * std::cos(phi) - y1 * std::sin(phi);                        // then swept about z
    s_target(1) = y1 * std::cos(phi) + s(0) * std::sin(phi);
    s_target(2) = z1;
}
// in target1_force_function:  wing_target(s_target, s, sweep_angle_pos, aoa_pos);  F = kappa_s * (s_target - X);
// in target2_force_function:  wing_target(s_target, s, sweep_angle_neg, aoa_neg);
```

- Both strokes use one formula, so position *and* velocity are continuous (ϕ and α are smooth); the cross-section is rotated, not sheared; $\mathbf{X} = \mathbf{s}$ at $t = 0$ when $\phi = 0$ and $\alpha = \frac{\pi}{2}$ (check: $y_1 = -s_1$, $z_1 = s_2 - p$: the wing's rest pose is the reference pose rotated by 90° about x, which is how the Trelis meshes were placed; shift `pivot` and the initial coordinate mapping of chapter 22 to taste).
- The two wings differ only by the signs of ϕ and of the pitch amplitude; one helper serves both. κ_s can then come down from 10^{13} to a value set by the drag and the grid (chapter 19, slide 26), which also loosens the time step.
- The insect-flight research codes next to the template (`IBAMR-Research-Codes/Insect Flight`) are this example's successors, with bristled and flexible wings; chapter 22 runs a 3D body in a tank.

THE 2018 TWO-CORAL EXAMPLE: THE TECHNIQUE, WITH A NEUTRAL BODY

The MATH 769 deck *2-IBFE Detailed Example using Target Points in 3D* (14 slides; private; its tether code was never shown and the coral model is not published) taught this sequence, which stands on its own:

1. Make the body in CAD (Fusion 360; the deck also lists Blender, SolidWorks, TinkerCAD): one polyp, say, or here a tapered cylinder.
2. Mesh it with hexahedra (Bolt, now Coreform Cubit; chapter 18) and export ExodusII.
3. Two bodies, one mesh file: read it twice, `mesh1.read("polyp.e")`, `mesh2.read("polyp.e")`, two parts.
4. Shift each copy with its own coordinate mapping ("`X = s + disp`"; slide 13's pattern), so that the two stand side by side.
5. Choose a motion and write it as a target position in the reference frame: the deck modelled it on a jellyfish bell ("Motion based on jellyfish bells; Angle relative to horizontal; Beta over time: contraction, expansion"): each point's target is its reference position rotated toward the body's axis by an angle $\beta(t)$ that grows with height, with $\beta(t)$ rising during a contraction phase and falling during a longer expansion phase.
6. Register the two mappings and the two target functions on parts 0 and 1, the same stress on both, and write two Exodus files; post-process both (the deck's sequel, chapter 28).

> In code this is the two-plate example (slide 13) with a 3D mesh and a bending target: $\mathbf{s}_{\text{target}} = R_{\beta(t)\,h(s)}(\mathbf{s} - \mathbf{s}_{\text{base}}) + \mathbf{s}_{\text{base}} + \mathbf{d}_{\text{part}}$, with $h(s)$ the normalized height and $\beta(t)$ a smooth pulse; exercise 5 (slide 28) builds it with two tapered cylinders.

UPSTREAM PATTERNS (1): EX6, A DISC AND A BEAM

`IBFE/explicit/ex6`: a disc (`build_sphere`) and a beam (`build_square`), both built in the program; each part its own tether and its own stress, each with its own file:

```
const string block_exodus_filename = app_initializer->getExodusIIFilename("block");
const string beam_exodus_filename = app_initializer->getExodusIIFilename("beam");
...
vector<MeshBase*> meshes(2);  meshes[0] = &block_mesh;  meshes[1] = &beam_mesh;
...
ib_method_ops->registerLagBodyForceFunction(block_tether_force_data, 0);
ib_method_ops->registerPK1StressFunction(block_PK1_stress_data, 0);
...
ib_method_ops->registerLagBodyForceFunction(beam_tether_force_data, 1);
ib_method_ops->registerPK1StressFunction(beam_PK1_stress_data, 1);
```

- The prefixes `"block"` and `"beam"` with no separator: with the default `output.ex2` these are `blockoutput.ex2` and `beamoutput.ex2`, the course's `mesh1output.ex2` pattern.
- `ex7`, which chapter 23 walks through, has the same shape: a lower wall (part 0) held by a tether alone and an upper wall (part 1) with a tether and its own stress.

UPSTREAM PATTERNS (2): EX8, THREE PARTS AND A SURFACE TETHER

`IBFE/explicit/ex8`: two blocks held by body-force tethers and a beam held by a *surface* tether on its long sides (chapter 19, slide 24):

```
vector<MeshBase*> meshes(3);
meshes[0] = &block1_mesh;  meshes[1] = &block2_mesh;  meshes[2] = &beam_mesh;
...
ib_method_ops->registerLagBodyForceFunction(block_tether_force_data, 0);
ib_method_ops->registerLagBodyForceFunction(block_tether_force_data, 1);   // the same function on two parts
...
ib_method_ops->registerLagSurfaceForceFunction(beam_tether_force_data, 2);
ib_method_ops->registerPK1StressFunction(beam_PK1_dev_stress_data, 2);
ib_method_ops->registerPK1StressFunction(beam_PK1_dil_stress_data, 2);
```

- A function can serve several parts (ex8's blocks; IBFE/08's stresses); a part can have functions no other part has (ex8's beam). The part argument, not the function, decides.
- Chapter 25 tours ex6 and ex8 with their inputs and results.

LOOKING AT SEVERAL PARTS IN VISIT

1. Open `viz_IB2d/dumps.visit`; **Add** → **Pseudocolor** → **Omega**; **Draw** (“make first”: the fluid database gives the time axis).
2. Open `viz_IB2d/beam_output.ex2` and add a **Mesh** plot; open `circle_output.ex2` and add a second **Mesh** plot. Each file is a separate database with its own active source; each needs its own plot.
3. Make the expression `dX = {dX_0, dX_1}` once (expressions are per session, and both files have the same variable names), and add **Operators** → **Transforms** → **Displace** with variable `dX` to both mesh plots. “Apply operators to all plots” ticked does both at once, but also hits the fluid plot; untick and apply twice, or tick, apply, and remove the operator from the fluid plot.
4. The time slider steps all three databases together as long as their numbers of states agree (81 here). If one part was written at a different interval, VisIt’s database correlation (chapter 28) lines them up by time.

- Scripted (`wsl/final/visit_ch21.sh`): a loop over the part files, `OpenDatabase`, `DefineVectorExpression`, `AddPlot("Mesh", "Mesh")`, `AddOperator("Displace")` per file; chapter 32.
- Colouring the two parts differently (two `MeshAttributes`), or colouring each by its own `F` or `U`, is chapter 28; measuring the gap (the plate’s top nodes against the disc’s bottom nodes) is a two-file Python script in chapter 32.

WHAT CAN GO WRONG

symptom	cause and fix
the second body has no stiffness, or no tether, or the wrong one	the part argument: `register*(data)` defaults to part 0; every part needs its own stress registrations (IBFE/08 registers the same two on 0 and 1)
one `.ex2` file, with one body in it	both writers given the same name: no prefix, or the same prefix; `getExodusIIFilename("beam_")` and `("circle_")`
`mesh1output.ex2` with no separator	the prefix is glued to the name; put the underscore in the prefix
a restarted run's `.ex2` files cannot be opened	`append(true)` on every rank; append on rank 0 only (slide 8)
`no matching function ...`	the 2015 `vector<Mesh*>`; the constructor takes
`IBFEMethod(..., std::vector<libMesh::Mesh*>&, ...)`	`std::vector<MeshBase*>` (chapter X3)
the post-processing writes one body	a `postprocess_data` with one mesh and one `EquationSystems` (the 2015 note); loop over parts
the two copies of one file sit on top of each other	no coordinate mapping, or both mappings the same; and the target functions must apply the mapping to `s` too (slide 13)
a force spike every half-stroke; `DT` has to shrink	kinematics that switch formula mid-cycle (slide 19); one formula, one pivot (slide 21); and $\kappa_s = 10^{13}$ is not needed
the bodies pass through each other	they cannot; what you see is two bodies' forces spread into the same cells when closer than two or three DX: refine, or add a contact penalty (slide 12)
the free body drifts away for no reason	momentum in a periodic box (slide 11); use a tank with walls (chapter 7)

WHAT THE OLDER DECKS HAD THAT IS NOW PART X

2015–2020	now
`Mesh mesh1(NDIM); vector<Mesh*> meshes`	`Mesh mesh1(init.comm(), NDIM);` `std::vector<MeshBase*>`
`AutoPtr<ExodusII_IO>`	`std::unique_ptr<ExodusII_IO>` (the 2020 deck's own correction)
`getExodusIIFilename("mesh1")` → `mesh1output.ex2`	a prefix with its separator: `beam_output.ex2`
`postprocess_data` with one mesh	a loop over parts
the 2015 deck's "…multipleplate2d"	`Example_IBFE_TetherForceMultiplePlate2D` in the 2016-11 set (slide 13)
the wings' two-pivot kinematics, "corrected equations forthcoming"	slide 21
Trelis trial version; "Trellis"	Coreform Cubit (chapter 18)
the two-coral deck's Fusion 360 / Bolt workflow and its images	the technique on slide 22; the images and the coral model stay private

EXERCISES

1. Run IBFE/08 with `TETHER_END_TIME = 2.0` and measure the gap every 0.5 from the two files. At what time is it smallest, and how many fine cells is that?
2. Give the disc a tether of its own (register a second body force on part 1 that holds it at its starting position with $\kappa_s = 10^4$). Does the plate reach it now?
3. Change the disc's material only: a copy of the two stress functions with $\mu = 1$, registered on part 1. How much does the disc deform at $t = 2$ with `TETHER_END_TIME = 2.0`?
4. Port the 2016 two-plate example to IBFE/08's program: read `IBFE_Mesh2D_128.mat` twice, register the two coordinate mappings and the two mirrored targets, and check that the plates stand 0.4 apart at $t = 0$ and 1.4 apart at $t = 1$.
5. The coral technique with a neutral body: mesh a tapered cylinder with gmsh (chapter 18), read it twice, shift the copies by ± 0.3, and bend each toward its own axis with $\beta(t) = \beta_{\max}\,\frac{1}{2}(1 - \cos 2\pi t/T)$ for a $T = 1$ cycle, the rotation angle growing linearly with height. Run in a $2 \times 2 \times 2$ box for one cycle.
6. Write the corrected wing kinematics into the two-wings program, run two cycles at `KAPPA_S = 1e6`, and compare the force on the wings (chapter 11) with the original code's at the half-strokes.

RULES OF THUMB

- A part is the index of a mesh in the vector you give `IBFEMethod`; every `register*` call and every output file carries it.
- Register the stress on every part; a part without a stress has no stiffness.
- One `ExodusII_IO` per part, one prefix per part with its separator, one `write_timestep` per part per dump; append on rank 0 only.
- Two files for two bodies; one file read twice plus coordinate mappings for two copies; subdomains for one body with two materials.
- A target function sees the file's `s`: apply the same mapping to it that placed the body.
- One formula for a prescribed motion, continuous in position and velocity; a rigid body is a rotation, not a shear.
- Bodies stop two or three cells apart; the film between them is real; a periodic box keeps the momentum you put in.

SUMMARY

1. Several meshes become parts of one `IBFEMethod`; each part gets its own registrations, data manager and ExodusII file, named by a prefix glued to the input's file name.
2. IBFE/08 drives a tethered plate at a free disc: the plate follows its target, coasts after release, and closes to six cells (three with a longer push) without touching; the disc rides the momentum the push left in the periodic box.
3. The course's two-mesh code went from `vector<Mesh*>` and `AutoPtr` (2015) to `MeshBase*` and `unique_ptr` (2020); its two-plate example placed one mesh twice with coordinate mappings, the pattern the 2018 corals used in 3D and that slide 22 teaches with a neutral body.
4. The 2020 wings are two parts with mirrored target kinematics and a 10^{13} tether; their formula switch at each half-stroke keeps the position continuous but not the velocity, and shears the thickness; a single rigid-rotation formula with one pivot fixes both.
5. In VisIt each part is its own database with its own mesh plot and displacement; restarts of multi-part runs follow the rank-0 rule.

Chapter 22 takes IBFE into 3D: the flow tank, a plate across a stream, the coordinate mapping in full, and what a 3D run costs.

RECORDINGS (LISTED, NOT INCORPORATED)

The 2020 Zoom recording of the IBFE lecture *Two Boundaries* and the 2018 MATH 769 session on the two-coral example are listed in the front matter's video list (chapter 0.1). The 2020 deck's two renders and the 2018 deck's six image slides are not reproduced; the figures here are from the shipped run.

REFERENCES AND CREDITS

- IBAMR 0.19.0: `include/ibamr/IBFEMethod.h` (the two constructors), `ibtk/src/utilities/AppInitializer.cpp` (`getExodusIIFilename`), `examples/IBFE/explicit/ex6`, `ex7`, `ex8`. https://github.com/IBAMR/IBAMR
- The examples of this series: IBFE/08 plate hits circle (`main.cpp`, `input2d`, `beam.e`, `circle.e`, README with the measured table); `examples/legacy/UNC-Examples/7-IBFE-TwoWings` (`main.C`, `input3d`, `pos_wing.e`, `neg_wing.e`); the course's `Example_IBFE_TetherForceMultiplePlate2D` (2016-11 set); `wsl/final/visit_ch21.sh`.
- Sources: *6-IBFE: Two Meshes* (UNC, 2015, edited 2018); *5-IBFE Lecture: Plans for June*, slides 5–8 (2020); *7-IBFE: Two Boundaries: Flapping wings and beam hits circle* (2020); *2-IBFE Detailed Example using Target Points in 3D* (MATH 769, 2018; technique only). The wing meshes and the beam and circle meshes were made with Trelis (2020).
- On clap-and-fling at low Reynolds number: L. A. Miller and C. S. Peskin, *When vortices stick: an aerodynamic transition in tiny insect flight*, J. Exp. Biol. 207 (2004) 3073–3088; and the successors in `IBAMR-Research-Codes/Insect Flight`.